ApplyTrack

MERN Project Specification and Build Handbook

Target: Junior developer / internship portfolio practice
Version: 1.0 | 19 October 2026
Project: A job application tracker built with React, Node.js, Express, and MongoDB.

Build a small application that you can explain, test, and change independently. The finish line is not a beautiful screenshot: it is a working flow from a browser form to a database and back, including failures.

Start here: Read sections 1-4, complete setup in section 5, then work through the build tickets in section 11. Use the data model and API contracts as your reference while coding. Configuration snippets are included; the feature implementations are your assignment.

Release boundary: Version 1 is a single-user local application using fictional records. A public portfolio deployment must be read-only. Accounts and private multi-user data are a later extension, not a hidden requirement for this sprint.

Handbook map

Sections	What you will find
1-3	Project brief, scope, pages, and acceptance criteria
4-6	Architecture, setup, and database rules
7-10	API contracts, backend workflow, React structure, and interface requirements
11-13	Build tickets, 14-session schedule, tests, and debugging
14-16	Safe deployment, portfolio handoff, and optional extensions
17-18	Glossary and official technical references

1. Project brief

1.1 The problem

A job seeker applies to several companies and needs one place to record each application, track its status, and find the details later. ApplyTrack solves that problem without attempting to become a recruitment platform.

1.2 The user

The first user is you, running the application locally. During development, use fictional companies and notes. A reviewer should be able to clone your repository, follow the README, and run the same project.

1.3 The assignment

Build a responsive website that lets the user create, read, update, and delete job applications. Add company/role search, status filtering, date sorting, a small summary dashboard, and clear loading and error states.

The project choices below are requirements for this exercise, not universal rules for every MERN application. The 14-session schedule is a suggested practice plan, not a promise of mastery or employment.

1.4 What you should demonstrate

You should be able to explain how a requirement becomes a component, API route, validation rule, database operation, and test. You should also be able to make one new change without copying an entire solution.

A good final demonstration includes a successful request, a rejected invalid request, persistence after refresh, and a short explanation of one bug you diagnosed.

2. Scope and definition of finished

2.1 Required for version 1

ID	Requirement	Evidence that it works
F01	Create an application	A valid form creates one record; it remains after refresh.
F02	List and view applications	The list and detail page show database records.
F03	Edit an application	Changed fields persist; untouched fields remain unchanged.
F04	Delete with confirmation	Cancel does nothing; confirm removes the record.
F05	Search and filter	Company/role search combines with one status filter.
F06	Sort	Newest/oldest application date ordering is consistent.
F07	Dashboard	Summary counts and five recent applications are correct.
F08	Validation and failures	Invalid input and failed requests produce useful feedback.
F09	Responsive and keyboard usable	Core flows work on a narrow screen and without a mouse.
F10	Reproducible handoff	README, environment examples, tests, and demo evidence exist.

2.2 Deliberately out of scope

Do not add login, payments, email reminders, file uploads, AI recommendations, a Kanban drag-and-drop board, Redux, microservices, or a complicated design system during the core sprint. Authentication is addressed as a separate extension in section 16.

Version 1 loads the complete, small practice collection. Search and sorting happen in React. There is no server pagination or claim that this design is suitable for a large dataset. Keep demo records to roughly 10-30 items; this is a practice recommendation, not a database limit.

2.3 Definition of finished

The required features work; the specified negative tests pass; the project starts from a clean clone; secrets are absent from Git; the production frontend build succeeds; the README states the single-user/read-only limitations; and you can trace create, update, and delete without reading a tutorial.

A feature is not finished merely because its button exists. It needs a working success path, failure behavior, and a repeatable check.

3. Pages and user experience

3.1 Route map

Browser route	Page	Required behavior
`/`	Dashboard	Show summary counts and the five most recent applications.
`/applications`	Applications	List, search, filter, sort, and link to details.
`/applications/new`	Add application	Display an empty form; save a valid record.
`/applications/:id`	Application details	Load one record; offer Edit and Delete actions.
`/applications/:id/edit`	Edit application	Load existing values into the shared form.
Any other path	Not found	Explain the problem and link back to Applications.

These are browser routes, not API endpoints. Browser pages render the interface; `/api/...` endpoints return JSON.

3.2 Dashboard

Display four cards: Total, Applied, Interviewing, and Offers. Total is the number of all records, including rejected and withdrawn applications. Each other card counts exactly its named status. The four displayed numbers are not supposed to add up to Total.

Show the five newest records by `appliedOn`, breaking ties by `createdAt` descending and then `_id` descending. The counts use the full collection, not the current list search. An empty database shows zeros and an Add application link. A failed request shows an error, not misleading zero counts.

3.3 Applications page

Include a heading, Add application link, search input, status select, sort select, result count, and records. Display company, role, status, application date, and View link for every record. Prefer a table on desktop and readable cards on a narrow screen.

Search is case-insensitive substring matching against company OR role. Trim the search query. Apply the status condition AND the search condition. An empty search means no text restriction. The All statuses option means no status restriction.

Default sort is newest application date first. Oldest reverses the complete ordering, including tie breakers. Never mutate the source state array when sorting; sort a copy. A no-match result offers Clear filters. A genuinely empty database offers Add your first application.

3.4 Add and edit forms

Use the same `ApplicationForm` component for both pages. Add starts with empty text values and status `applied`; the user chooses the application date. Edit starts with the saved values, not with a blank form that briefly overwrites them.

On submit, show Saving and disable repeated submissions while the request is pending. Keep typed values when validation or a request fails. Display field errors near their inputs and a form-level error for network/server problems. Navigate to the saved detail page only after the server confirms success.

A Cancel link returns without saving. A full unsaved-changes warning is optional; do not let it block completion of the main flow.

3.5 Details and deletion

Show every saved field, plus readable created/updated timestamps. Empty notes or job links should display a friendly placeholder or be omitted. An unknown record shows Application not found rather than a blank page.

Deletion must name the company in its confirmation. Use a browser confirmation initially; a custom accessible dialog is an optional refinement. Cancel sends no request. On success, navigate to Applications. On failure, keep the existing details visible and show the error.

3.6 Simple layout sketch

```
ApplyTrack       Dashboard | Applications | Add application

Applications                              [Add application]
[Search company or role...] [All statuses v] [Newest v]

Showing 8 of 12 applications
Company          Role               Status        Action
Northstar Labs   Frontend Intern    Applied       View
Cedar Studio     Junior Developer   Interviewing  View

[Loading / error / empty message occupies this content area]
```

This sketch defines information placement, not a required visual style. Use readable spacing and consistent buttons rather than spending several days designing a logo.

4. Architecture and project structure

4.1 The request journey

```
Browser event
  -> React form handler
  -> API helper using fetch
  -> Express route
  -> Input validation
  -> Controller
  -> Mongoose model
  -> MongoDB
  -> JSON response
  -> React success or error state
```

For this project, the controller can call the model directly. Separate service and repository layers are not required. Keep `app.js` separate from `server.js`: one configures Express; the other connects to MongoDB and starts listening. This makes the boundaries easier to explain and test.

4.2 Tools

Use JavaScript, React with Vite, React Router in declarative mode, Express 5, MongoDB, Mongoose, plain CSS or CSS Modules, and the browser Fetch API. React Router's current declarative installation uses the `react-router` package. [2, 3]

Use a current patched Node.js 24 LTS release for this handbook. Node's release page lists 24 as LTS at the time of writing; confirm supported versions when setting up later. Vite also publishes its current minimum Node requirements. Commit both package lockfiles to record the versions actually installed. [1, 2]

4.3 Suggested folders

```
applytrack/
  client/
    src/
      api/applications.js
      components/
        AppLayout.jsx
        ApplicationForm.jsx
        ApplicationList.jsx
        StatusBadge.jsx
        FeedbackMessage.jsx
      pages/
        DashboardPage.jsx
        ApplicationsPage.jsx
        NewApplicationPage.jsx
        ApplicationDetailsPage.jsx
        EditApplicationPage.jsx
        NotFoundPage.jsx
      utils/applicationSelectors.js
      App.jsx
      main.jsx
      styles.css
    vite.config.js
    package.json
    package-lock.json
  server/
    src/
      config/db.js
      models/Application.js
      routes/applicationRoutes.js
      controllers/applicationController.js
      validators/applicationInput.js
      middleware/errorHandler.js
      middleware/readOnlyGuard.js
      app.js
      server.js
    tests/validation.test.js
    .env.example
    package.json
    package-lock.json
  docs/
    api.md
    test-results.md
    decisions.md
  .gitignore
  README.md
```

Create folders as you need them; empty files are not progress. Avoid storing database calls inside React or DOM manipulation inside controllers.

5. Local setup

5.1 Prerequisites

Install the supported Node version, Git, an editor, and either a local MongoDB instance or access to a MongoDB deployment. An Atlas connection needs a database user and an appropriate network access entry; these are distinct from simply signing into the Atlas website. Keep access as narrow as practical. [1, 8]

The following commands are intended for Bash or Git Bash. On Windows PowerShell, use the equivalent directory commands and `curl.exe` when its `curl` alias behaves differently. Create files using your editor.

```
node --version
npm --version
git --version
mkdir applytrack
cd applytrack
git init
npm create vite@latest client -- --template react
cd client
npm install
npm install react-router
cd ..
mkdir server
cd server
npm init -y
npm install express@5 mongoose
npm pkg set type=module
cd ..
```

Vite scaffolds the frontend only. The remaining server files and features do not exist until you implement them. [2]

5.2 Server scripts

Merge these entries into `server/package.json`; keep the generated dependencies. Create the referenced files before trying to run their scripts.

```
{
  "type": "module",
  "scripts": {
    "dev": "node --watch --env-file=.env src/server.js",
    "start": "node src/server.js",
    "test": "node --test tests/validation.test.js"
  }
}
```

Node provides watch mode, environment-file loading, and a built-in test runner. `dev` reads a local environment file; `start` expects variables supplied by the deployment environment. [4, 13]

5.3 Environment and Git

Create `server/.env.example` with these non-secret local defaults, then copy it to `server/.env` in your editor. Replace the URI locally when using a remote database.

```
PORT=5000
HOST=127.0.0.1
MONGODB_URI=mongodb://127.0.0.1:27017/applytrack_dev
NODE_ENV=development
DEMO_READ_ONLY=false
```

Put this in the root `.gitignore` before your first commit:

```
**/node_modules/
**/dist/
.env
.env.*
!.env.example
coverage/
*.log
```

Keep connection strings only on the server. Vite exposes `VITE_` variables to client code, so that prefix must never contain database credentials or other secrets. If a secret reaches Git, rotate it; deleting its latest appearance alone is not a remedy. [10, 21]

5.4 Development proxy

Update the generated `client/vite.config.js`:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true,
    proxy: {
      '/api': 'http://127.0.0.1:5000'
    }
  }
});
```

Use relative requests such as `fetch('/api/applications')`. The Vite development server forwards matching `/api` requests to Express. This proxy is a development setting, not something automatically installed on your production host. Production will use one origin for frontend and API. [5]

After building the health route, start the server and client in separate terminals from their respective folders using `npm run dev`. Open the Vite address and verify that both direct and proxied health requests work.

6. Database and validation specification

6.1 Application fields

Collection name: `applications`. There is no `users` collection in version 1.

Field	Stored form	Rule
`_id`	MongoDB ObjectId	Server/database generated; response uses its string form.
`company`	String	Required; trim; length 2-100.
`role`	String	Required; trim; length 2-100.
`status`	String	One allowed lowercase value; default `applied` on create.
`appliedOn`	String	Required, actual calendar date in `YYYY-MM-DD` form.
`jobUrl`	String	Optional; default empty; otherwise HTTP/HTTPS URL, max 2,048.
`notes`	String	Optional; default empty; trim; maximum 2,000.
`createdAt`	Date	Server managed; ISO timestamp in JSON.
`updatedAt`	Date	Server managed; ISO timestamp in JSON.

Allowed statuses are `applied`, `interviewing`, `offer`, `rejected`, and `withdrawn`. Display friendly capitalized labels, but send the lowercase API values.

`appliedOn` is deliberately a date-only string, not a timestamp. Validate both its shape and actual calendar validity: `2026-02-31` must fail. Version 1 imposes no additional past/future date rule. Treat `createdAt` and `updatedAt` as timestamps; do not substitute them for the application date.

Allow repeated applications to the same company and role. Do not add a uniqueness constraint for that pair. Status can change between any allowed values; a real-world hiring state machine is outside this sprint.

6.2 Request validation rules

Accept a plain JSON object, not `null`, an array, or a primitive. Validate raw types before relying on Mongoose casting: a numeric company value is not acceptable even when it could be converted to a string.

Only `company`, `role`, `status`, `appliedOn`, `jobUrl`, and `notes` are writable. Reject unknown fields, including `_id`, `ownerId`, `createdAt`, `updatedAt`, and MongoDB operator objects. Build the database operation from approved values rather than passing the entire request body through.

On POST, require company, role, and appliedOn. Supply defaults for missing optional fields. On PATCH, accept only supplied fields and require at least one writable field. A PATCH containing only status must not erase notes or change the application date. Empty optional strings clear those fields; `null` is invalid.

6.3 Suggested validator interface

```
validateApplicationInput(body, { partial: false });
// -> { value: normalizedAllowedFields, errors: fieldErrors }

validateApplicationInput(body, { partial: true });
// -> same shape, but only supplied fields are validated/returned
```

This is an interface to implement, not a built-in function. `errors` is an object keyed by field name; use `_form` for a body-level problem. Only use `value` when there are no errors. Write this function independently of Express so unit tests do not require a running server.

Use Mongoose schema validation as a second boundary. A simple edit path is find document, assign approved fields, then `save()`. Mongoose runs validation on save; query-based update validators have separate behavior and are not on by default. [7]

7. API contract

7.1 Shared rules

Base path: `/api`. Successful reads/writes return JSON inside `data`, except successful DELETE, which returns 204 with no body. Errors return an `error` object. POST and PATCH require JSON content type.

Use a 32 KB JSON body limit. Return 400 for invalid fields, malformed JSON, or a malformed ID; 404 for a well-formed ID that does not exist; 413 for an oversized body; and 415 for a non-JSON POST/PATCH. In public read-only mode, recognized mutation requests return 403 before body validation.

Use 503 for a recognized database-unavailable condition and 500 for an unexpected internal failure. The public message must not expose a stack trace, URI, password, or raw database error. The exact codes/messages below are this project's chosen contract.

7.2 Endpoint summary

Method and endpoint	Purpose	Success
`GET /api/health`	Liveness and public demo mode	200

Method and endpoint	Purpose	Success
`GET /api/applications`	All records in the small collection	200
`GET /api/applications/:id`	One record	200
`POST /api/applications`	Create one record	201
`PATCH /api/applications/:id`	Update supplied fields	200
`DELETE /api/applications/:id`	Delete one existing record	204

The list endpoint has no query parameters in version 1. Browser search, status, and ordering are applied locally after retrieval. Return the list in the default newest-first ordering defined in section 3.

7.3 Health

```
{
  "data": {
    "status": "ok",
    "readOnly": false
  }
}
```

This endpoint reports that the process can respond; it is not a guarantee that every database operation will succeed. It lets the client show the read-only banner. Until this setting loads successfully, keep mutation controls unavailable and show a configuration-loading/error state.

7.4 Create

Request body for `POST /api/applications`:

```
{
  "company": "Northstar Labs",
  "role": "Frontend Developer Intern",
  "status": "applied",
  "appliedOn": "2026-10-14",
  "jobUrl": "https://example.com/jobs/frontend-intern",
  "notes": "Submitted the portfolio and resume."
}
```

Illustrative 201 response:

```
{
  "data": {
    "_id": "507f1f77bcf86cd799439011",
    "company": "Northstar Labs",
    "role": "Frontend Developer Intern",
    "status": "applied",
    "appliedOn": "2026-10-14",
    "jobUrl": "https://example.com/jobs/frontend-intern",
    "notes": "Submitted the portfolio and resume.",
    "createdAt": "2026-10-14T10:00:00.000Z",
    "updatedAt": "2026-10-14T10:00:00.000Z"
  }
}
```

The example ID and timestamps are illustrative, not values to hardcode. Set a `Location` header pointing to `/api/applications/<new-id>`. Do not create a record until validation passes.

7.5 List and detail

An empty list returns `{"data": [], "meta": {"count": 0}}`. A populated response puts complete application objects in the data array and the array length in `meta.count`. Because there is no pagination, this count is the total returned collection size.

A successful detail request uses the same single-object envelope shown in the create response. Do not wrap it in an array. Omit internal Mongoose version fields such as `__v` from the public representation.

7.6 Update and delete

Example PATCH body:

```
{
  "status": "interviewing",
  "notes": "Initial interview arranged."
}
```

Return the complete updated object, including the refreshed `updatedAt`. Keep `createdAt` unchanged. Return 404 when the target does not exist.

DELETE returns an empty 204 response. Do not try to parse that response as JSON in the client. Repeating deletion of the now-missing record returns 404 under this contract.

7.7 Error shapes

Validation failure, 400:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "fields": {
      "company": "Enter a company name of 2-100 characters."
    }
  }
}
```

Other errors may omit `fields`. Use `INVALID_ID`, `NOT_FOUND`, `INVALID_JSON`, `PAYLOAD_TOO_LARGE`, `UNSUPPORTED_MEDIA_TYPE`, `DEMO_READ_ONLY`, `SERVICE_UNAVAILABLE`, or `INTERNAL_ERROR` as appropriate. A generic internal message can be: Something went wrong. Please try again.

7.8 Manual API workflow

Start with the health and empty-list requests. Create a valid record and copy its returned ID; use that ID for detail, update, and delete. Repeat the list request after each change. Then run invalid-body and nonexistent-record cases before connecting React.

```
curl -i http://127.0.0.1:5000/api/health
curl -i http://127.0.0.1:5000/api/applications
```

Use an API client for JSON request bodies when shell quoting becomes distracting. Save the request bodies and observed results in `docs/test-results.md`.

8. Backend implementation workflow

8.1 Startup responsibilities

In `server.js`, validate environment values, connect to MongoDB, and only then start listening. Exit with a clear non-secret message when essential configuration or the connection is unavailable. Use `HOST=127.0.0.1` locally. A deployed web service generally needs the host interface and port required by its platform.

Treat `DEMO_READ_ONLY` as a boolean only after comparing its string value explicitly. Reject values other than `true` or `false`. For this no-auth version, refuse production startup unless read-only mode is enabled. Do not quietly make a public writable app the fallback.

8.2 Middleware order

Configure shared settings; mount the server-enforced read-only guard for API mutations; require JSON where appropriate; parse JSON with the size limit; register health/application routes; register a JSON API not-found handler; then add production static assets and browser-route fallback. Put the error handler after the routes it handles.

Express error middleware has four arguments. Express 5 forwards rejections from returned/awaited asynchronous route handlers, but unrelated callback/background failures still need appropriate handling. [6]

8.3 Controller responsibilities

Keep controllers small. A create handler validates, calls the model, maps the result to the public response, and returns 201. A detail handler validates ID syntax, looks up the record, distinguishes missing from invalid, and returns 200 or 404.

An update handler validates both ID and partial body, finds the record, copies only approved values, saves, and returns the updated representation. A delete handler checks ID, deletes the matching record, and returns 204 only when something existed to delete.

Before database calls, treat an ID as valid only when it is a string of 24 hexadecimal characters. Do not allow malformed ID errors to become accidental 500 responses.

8.4 Build one path before the rest

First make GET return a hardcoded array. Next replace it with a database read. Add POST and prove persistence. Only then build update and delete. Hardcoded data is a temporary checkpoint, not an acceptable final data source.

Do not log complete notes, request bodies, or connection strings. For debugging, a method, path, sanitized error description, and request identifier are normally enough for this exercise.

9. React implementation workflow

9.1 State ownership

Keep list data, loading/error status, search text, status selection, and sort selection in the Applications page. Compute the visible list from those values instead of storing a second editable copy. The dashboard can fetch the same collection when mounted and derive its own summary.

Keep form values, field errors, and submission state in the form/page pair. Pass initial values and an `onSubmit` function to the reusable form. React's guide recommends identifying minimal state and computing values that can be derived from existing state. [9]

Do not add global state solely because the project has more than one page. For this version, refetching when a page opens is an acceptable choice. Document that choice rather than pretending to have implemented an advanced client cache.

9.2 API helper interface

Implement these functions in `client/src/api/applications.js`:

```
listApplications({ signal } = {});
getApplication(id, { signal } = {});
createApplication(values);
updateApplication(id, changes);
deleteApplication(id);
```

Read helpers can accept an AbortSignal. List resolves to the data array; detail/create/update resolve to the data object; delete resolves without a value. Convert unsuccessful responses to an error containing `status`, `code`, `message`, and optional `fields`.

Check `response.ok`: Fetch does not reject just because the server returns an HTTP error. Handle network failures separately, and do not call `response.json()` for a 204 response. [11]

9.3 Effects and mutations

Use page-load fetching for read operations. Clean up or ignore stale read responses when a component unmounts or an ID changes. Treat an intentionally aborted read as cancellation, not a user-facing failure. Trigger POST/PATCH/DELETE from user event handlers, not from an effect that runs when a page mounts. [11, 19]

For the first implementation, wait for the server before changing the displayed result. Optimistic updates are not required. After create/edit, navigate to details and load the confirmed record. After deletion, navigate to the list and reload it.

9.4 Component boundaries

`AppLayout` owns navigation and the main content area. Page components own page-level fetching and state. `ApplicationForm` owns reusable form markup and interaction. `ApplicationList` renders records. `StatusBadge` maps known values to readable labels. `FeedbackMessage` standardizes loading/empty/error presentation.

The same API field names must be used across form values, validation errors, controllers, and response mapping. Avoid switching between `company`, `companyName`, and `organization` halfway through implementation.

10. Interface and quality requirements

10.1 Required visual states

Every data-driven page needs loading, success, empty or not-found, and failure behavior. Every mutation needs idle, submitting, success, and failure behavior. A loading spinner by itself is not a replacement for an error message.

On error, provide a Retry action for reads. When retrying, do not show an empty-database message until a successful response proves there are no records. A submitted form must not clear itself after a failed save.

10.2 Accessibility and mobile checks

Use real links for navigation and buttons for actions. Give every input a visible associated label. Connect field errors to their inputs; keep a clear focus indicator; make error feedback available to assistive technology; and do not communicate a status only through color. The MDN form guide illustrates explicit label/control associations. [12]

Use one clear page heading and semantic form/table markup. Choose descriptive actions such as View application instead of an unexplained icon. At a narrow viewport around 360 pixels, ensure fields, action buttons, and messages remain readable without horizontal page scrolling.

Render notes as plain text, not raw HTML. Validate external job URLs and permit only HTTP or HTTPS. Open external links with sensible browser security attributes when using a new tab.

10.3 Project housekeeping

Run the frontend lint script provided by the scaffold and fix meaningful warnings. Remove unused template assets. Use descriptive filenames, keep functions reasonably focused, and explain non-obvious decisions in

short comments. Do not spend the sprint reformatting code that already works while core behavior is missing.

Record decisions such as date-only storage, local filtering, and no-auth/read-only scope in `docs/decisions.md`.

11. Build tickets and 14-session schedule

11.1 Ticket workflow

For each feature, write: requirement, expected input/output, affected files, plain-English steps, failure cases, and evidence of completion. Implement the smallest checkable step, test it, then commit a working checkpoint.

Here is a complete example ticket:

Ticket AT-03: Create an application. A user can submit a company, role, and date and see the saved record. A blank company produces a field error and creates nothing. A network failure keeps the form values. Repeated clicks during a pending request do not send another request.

Affected files: validator, model, route, controller, API helper, NewApplication page, ApplicationForm, and tests. First prove POST in an API client; then connect the form. Verification: note the returned ID, reload its detail page, restart the server, and load it again.

Suggested commit: `feat: create validated applications from the form`.

11.2 Build order

Ticket	Work	Exit check
AT-01	Scaffold and health endpoint	Direct and proxied health requests succeed.
AT-02	Model and list endpoint	MongoDB records, including an empty list, are returned.
AT-03	Create end to end	Form submission persists and handles invalid input.
AT-04	Detail page	Known, malformed, and missing IDs behave correctly.
AT-05	Edit end to end	Partial changes persist without erasing other fields.
AT-06	Delete end to end	Confirmation, cancellation, success, and failure work.
AT-07	Search/filter/sort	Combined conditions and tie ordering match the spec.
AT-08	Dashboard and mobile	Counts are correct and all pages work narrowly.
AT-09	Tests and fixes	Negative cases and build checks pass.
AT-10	Handoff	README, recording, and optional safe deployment are complete.

11.3 Suggested sessions

Assume about two to three focused hours per session. Repeat a session when necessary. Deployment or unfamiliar database setup may require additional time; do not rush failures into the next milestone.

Session	Main objective	Evidence to save
1	Read the brief; sketch pages; practise filtering/updating arrays.	Page sketch and small working JS functions.
2	Scaffold folders, scripts, environment, and health endpoint.	Health response and first commit.

Session	Main objective	Evidence to save
3	Implement model, raw input validator, and database connection.	Unit checks for valid and invalid fields.
4	Implement list/create/detail endpoints and test them manually.	Saved API examples and returned IDs.
5	Build navigation and static React list/form pages.	Screenshots using clearly temporary fixtures.
6	Connect list and detail reads; handle load/error/empty states.	Database-backed pages and error screenshot.
7	Connect create; verify the complete request journey.	Refresh/restart persistence evidence.
8	Implement edit API and reusable edit form.	Before/after values and preserved fields.
9	Implement deletion and confirmation.	Cancel and delete test results.
10	Add search, status filter, and ordering.	Combined-filter and tie-order test results.
11	Add dashboard and responsive/keyboard refinements.	Correct counts and narrow-screen views.
12	Run negative cases; automate validator tests; fix bugs.	Test output and documented fixes.
13	Clean-clone check, README, recording; optional read-only deployment.	Reproduction instructions and demo evidence.
14	Rebuild a smaller create/list path without the original code.	Notes on what you could reconstruct independently.

11.4 Daily practice rule

Begin by choosing one observable outcome, not a vague goal such as finish backend. Spend most of the session coding. Close the tutorial solution; use documentation to answer specific questions. End with a tested commit and a note describing the next smallest task.

When asking for help, share the requirement, your code, expected result, actual result, and checks already performed. Ask for the first incorrect assumption or one hint before asking for a replacement implementation.

12. Testing and acceptance matrix

12.1 Unit tests

Start with the raw input validator because it can run without MongoDB or React. Test required fields, trimming, length boundaries, every allowed status, invalid types, date validity, URL protocol, empty PATCH, and partial-update behavior.

Example test for the validator interface you will implement:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { validateApplicationInput } from
  '../src/validators/applicationInput.js';

test('rejects a blank company', () => {
  const result = validateApplicationInput({
    company: '   ',
    role: 'Frontend Intern',
    appliedOn: '2026-10-14'
  }, { partial: false });

  assert.ok(result.errors.company);
});
```

Node's built-in runner supports tests through `node:test` and execution with `node --test`. The included package script targets this one test file; extend it deliberately when you add more. [13]

12.2 Acceptance cases

ID	Action or input	Expected result
T01	Load an empty database	200 with an empty array; friendly empty interface.
T02	Create valid required fields	201; defaults applied; one record created.
T03	Submit whitespace-only company	400 field error; no record created.
T04	Submit object/number instead of company text	400; no implicit conversion accepted.
T05	Submit unknown status or unknown field	400; no database change.
T06	Submit `2026-02-31`	400 date error.
T07	Submit a `javascript:` job URL	400 URL error.
T08	Submit invalid JSON / wrong content type	400 / 415 with the documented error shape.
T09	Submit a JSON body larger than 32 KB	413; no record created.
T10	Refresh after creating a record	The record remains.
T11	Request a malformed ID	400, not 500.
T12	Request a valid but nonexistent ID	404 with a useful interface message.
T13	PATCH only status	Status changes; company/date/notes remain unchanged.
T14	PATCH an empty object	400 body-level error.
T15	Cancel deletion	No DELETE request; record remains.
T16	Confirm deletion, then fetch that ID	204, followed by 404.
T17	Search mixed-case company/role and filter status	Only records matching both conditions appear.
T18	Choose sort with equal application dates	Documented tie ordering is respected.
T19	Apply a filter matching nothing	No-match message, Clear filters, no data loss.
T20	Stop the server during use	Read/save failure is shown; form input is retained.
T21	Click Save repeatedly while pending	One in-flight submission from that form.
T22	Navigate directly to a browser detail route	Page loads, including after deployed refresh.

ID	Action or input	Expected result
T23	Navigate by keyboard on a narrow viewport	Core actions and labels remain usable.
T24	Call mutations directly on public demo API	403 even without using the frontend.
T25	Start production with read-only mode disabled	Startup refuses to expose this no-auth version.

Record pass/fail, date, environment, and a short evidence note. Do not write all tests pass without executing the relevant checks. T24-T25 are release-gate tests for a public demo, not requirements to claim that a local-only demo is deployed.

12.3 Test data and isolation

Use an explicitly separate test database, for example `applytrack_test`, for automated database tests. Never point destructive setup/cleanup at your development or deployed database. Add a guard that refuses database cleanup unless the configured database name matches the dedicated test name.

As a later improvement, automate API tests around an isolated app instance and test database. For the core sprint, automated validator tests plus documented manual end-to-end checks are the minimum; do not label manual checks as automated coverage.

13. Debugging playbook

13.1 Locate the first broken boundary

Use this order: event handler, outgoing request, response status/body, server route, validation, database operation, and React state after the response. Write down the first point where the actual value differs from the expected value.

The browser Network panel exposes request URLs, payloads, status codes, and responses. Use it before guessing that the database is broken. [20]

13.2 Frequent symptoms

Symptom	First checks
Form reloads the page	Form submit handler, `preventDefault`, button type.
Request reaches the wrong address	Relative `/api` path, Vite proxy, server port.
Response parsing sees HTML	Wrong path or SPA fallback catching an API request.
Server gets no expected body	JSON header, parser order, actual outgoing payload.
Saved record is absent on screen	Response shape, navigation/refetch, state update.
Data disappears on restart	Real database persistence versus an in-memory array.
PATCH erases unrelated fields	Optional defaults incorrectly applied to a partial update.
Database connection fails	Server URI, database user, network access, service availability.
HTTP error looks like success	Missing `response.ok` check in the API helper.
Detail page shows an older record	Stale asynchronous read after a route parameter change.

These are investigation prompts, not diagnoses. Confirm each possibility with evidence. Change one thing, repeat the same test, and remove temporary logs afterward.

13.3 Debug note template

```
Expected:
Actual:
First incorrect boundary/value:
Evidence:
Change made:
Retest result:
What I learned:
```

Keep three real examples for your project explanation. An honest account of finding and fixing a bug is more useful than a claim that the project had no bugs.

14. Safe deployment plan

14.1 Choose the correct release

A local demonstration plus a short recording is an acceptable handoff for this exercise. A live link is optional. Do not upload the no-auth writable API and invite people to store private information.

For a live portfolio demo, publish fictional records in read-only mode. Show a visible banner and disable mutation controls. The Express guard must still reject direct mutation requests; hiding a button is not an authorization boundary. Authorization controls belong on the server. [14]

Core guard logic to adapt into `readOnlyGuard.js`:

```
export function readOnlyGuard(readOnly) {
  return (req, res, next) => {
    const mutation = ['POST', 'PUT', 'PATCH', 'DELETE']
      .includes(req.method);

    if (readOnly && mutation) {
      return res.status(403).json({
        error: {
          code: 'DEMO_READ_ONLY',
          message: 'This public demo is read-only.'
        }
      });
    }
    next();
  };
}
```

Mount this for `/api` before application mutation handlers. Keep the startup rule from section 8: this version must refuse production startup with read-only mode off. Populate the fictional demo database privately; do not publish a seed/reset endpoint. Use a database credential limited to reading the demo collection where supported.

14.2 Same-origin deployment

Build the React frontend and serve `client/dist` from the Express application. Keep API routes and the JSON `/api` not-found handler ahead of the browser fallback. In Express 5, a fallback matching the root and nested browser paths can use `/{*splat}`; older unnamed wildcard examples are incompatible. [16]

Implementation outline for `server/src/app.js`, after API routes and API 404 handling:

```
// Imports at the top of app.js:
import path from 'node:path';
import { fileURLToPath } from 'node:url';

// express and app must already be defined.
if (process.env.NODE_ENV === 'production') {
  const clientDist = fileURLToPath(
    new URL('../../client/dist/', import.meta.url)
  );
  app.use(express.static(clientDist));
  app.get('/{*splat}', (req, res) => {
    res.sendFile(path.join(clientDist, 'index.html'));
  });
}
```

Keep this conditional block before the final error handler. Test an unknown `/api` URL: it must return a JSON 404 rather than the frontend HTML. The Vite development proxy is not needed when both frontend and API share the deployed origin.

14.3 Example web-service configuration

One deployment option is a Node web service on Render; its official guide explains connecting a repository and specifying build/start commands. Check the provider's current limits and pricing before creating a service. This handbook does not promise a permanently free hosting plan. [17]

For this two-folder repository, leave the service root at the repository root and use:

```
# Build command
npm ci --prefix client && npm run build --prefix client && npm ci --prefix server

# Start command
npm start --prefix server
```

Configure the supported Node version consistently with local development, `NODE_ENV=production`, `HOST=0.0.0.0`, `DEMO_READ_ONLY=true`, and a server-side demo `MONGODB_URI`. Read the platform-provided `PORT` in your server. Keep environment secrets in the host's environment settings, not in the frontend build.

For Atlas, permit the deployment's required outbound addresses rather than treating unrestricted network access as the default. Use HTTPS and review the framework's production security guidance before exposing the service. [8, 15]

14.4 Release gate

Before sharing a link, verify page refresh on nested routes, read-only API enforcement, production configuration checks, fictional-only data, no exposed secrets, correct API 404 responses, and a successful build. Review dependency warnings and document any unresolved limitations honestly.

15. Portfolio handoff and self-assessment

15.1 README contents

Explain what the application does, the intended user, completed features, screenshots, stack, prerequisites, exact local setup commands, environment variables, test commands, and known limitations. Include the architecture/request-flow diagram and link to your API contract and test record inside the repository.

State explicitly that the public demo is read-only and the local version has no accounts. Include a recording of local create/edit/delete so a reviewer can see those features even when the public demo cannot modify data.

Test the README from a clean clone in a new folder. Use `npm ci` with the committed lockfiles, create the environment file from its example, and follow only your written instructions. Fix every missing step.

15.2 Suggested three-minute demo

First show the problem and dashboard. Create one application, refresh it, edit its status, search for it, then delete it with confirmation. Show one invalid submission and its error. Finish with the request flow and one real bug you fixed.

Do not imply that a public read-only link has login or multi-user privacy. Describe what exists, not the extension backlog.

15.3 Resume description after completion

Adapt this only to features you actually built:

Built a MERN job application tracker with validated CRUD APIs, reusable React forms, search/status filtering, and responsive pages. Documented API contracts and test cases, and demonstrated database persistence and failure handling.

Add a deployment or automated-testing claim only when it is true. Do not invent user counts, performance improvements, or test coverage percentages.

15.4 Questions you should be able to answer

Explain the complete create request journey. Why is validation repeated on the server? What distinguishes a malformed ID from a missing record? How does PATCH avoid erasing fields? Why does Fetch need a status check? Where does React state live? How do you handle a stale request? Why are secrets server-only? What keeps the public demo read-only? What would change for many users or thousands of records?

A useful self-check is to implement a small change, such as a new location field, without the original solution open. Update the model, validator, API example, form, detail page, and tests. The important result is a consistent change across the application.

16. Extensions after the core sprint

16.1 Choose one small extension first

Add a location field, a follow-up date, server-side search/pagination, or an automated API test suite. Do not add all four at once. Write the acceptance criteria before changing code and preserve the existing tests.

If you add pagination, move global counts to an all-records backend calculation rather than counting only the visible page. Update the API response contract and empty/loading behavior deliberately.

16.2 Multi-user version: separate security milestone

Accounts are not just a login page. Add an authenticated identity, an owner reference on each application, and server-side ownership checks on every list/read/update/delete operation. Derive the owner from the authenticated session, never from a trusted-looking `ownerId` in the request body. Test that user A cannot access user B's record even with its ID. [14]

A possible follow-on route set is register, login, logout, and current user, but select a maintained authentication approach before implementation. For cookie-based sessions, address secure cookie settings, session lifecycle, server-side invalidation, and CSRF protection appropriate to the deployment. Follow the authentication/session provider's current documentation. [18]

Do not remove the public read-only gate until the replacement access controls and cross-user tests are implemented. Password storage, account recovery, abuse prevention, and private-data handling need their own specification; they are not completed by this version-1 handbook.

17. Quick glossary

Term	Meaning in this project
CRUD	Create, read, update, and delete application records.
Endpoint	A server path plus HTTP method, such as POST `/api/applications`.
Contract	The request, response, and error behavior both sides agree to use.
Middleware	A function that runs within the Express request/response chain.
Validation	Checking input against the project's permitted shape and values.
Persistence	A record remains in the database beyond a page refresh or server restart.
Acceptance criterion	An observable condition that proves a feature behaves as specified.
Regression	A change breaks behavior that previously worked.
Read-only demo	Public visitors can view fictional data but cannot alter it.
Authorization	Checking whether an identified user may perform a particular action.

18. Official technical references

The requirements, field limits, endpoint shapes, and schedule in this handbook are project design choices. The sources below support the referenced tool behavior and security guidance. Consult the current page when an installed version behaves differently.

[1] Node.js - Release status and supported versions.

[2] Vite - Getting started, templates, and Node requirements.

[3] React Router - Declarative installation.

[4] Node.js - Command-line options, watch mode, and environment files.

[5] Vite - Development server proxy.

[6] Express - Error handling.

[7] Mongoose - Validation and update-validator behavior.

[8] MongoDB Atlas - Connection prerequisites.

[9] React - Thinking in React.

[10] Vite - Environment variables and client exposure.

[11] MDN - Using the Fetch API.

[12] MDN - Your first form.

[13] Node.js - Built-in test runner.

[14] OWASP - Authorization Cheat Sheet.

[15] Express - Production security guidance.

[16] Express - Migration to Express 5 and path syntax.

[17] Render - Deploy a Node Express application.

[18] OWASP - Session Management Cheat Sheet.

[19] React - Synchronizing with Effects and fetching-data cleanup.

[20] Chrome DevTools - Network panel.

[21] GitHub - Removing sensitive data and rotating exposed credentials.